PP-Configuration for Enterprise Server Applications and Agent/Application Component(s)

Table of Contents

Acknowledgements

This PP-Configuration was developed by the iTC for Application Software international Technical Community (iTC) also known as AppSW-iTC with representatives from Industry, Government agencies, Common Criteria Test Laboratories, and members of academia.

Revision History

Table 1. Revision history

Version	Date	Description
1.0	2022-04-06	Initial Release
1.0e	2024-02-15	Incorporated feedback received following initial release.
2.0	2026-07-21	Draft-review updates.

1. Introduction

1.1. PP-Configuration Overview

The purpose of a PP-Configuration is to combine Protection Profiles (PPs) and PP-Modules for various technology types into a single configuration that can be evaluated as a whole.

This PP-Configuration is for enterprise server applications and their agent or application component(s). It provides the enforceable PP-Configuration path for distributed application software whose component relationships satisfy the Server and Agent Module control model, including applicable server-agent deployments, clustered server deployments, and microservices architectures composed of multiple application payload components.

1.2. PP-Configuration Reference

This PP-Configuration is identified as follows:

- PP-Configuration for Enterprise Server Applications and Agent/Application Component(s), Version 2.0, 2026-07-21
- As a shorthand reference, it can be identified as "CFG_APP-Server-Agent_V2.0"

1.3. PP-Configuration Components

This PP-Configuration includes the following components:

Table 2. PP-Configuration Components

[base PP]	cPP_APP_SW_V2.0
[PP-Module 1]	MOD_Server_v2.0
[PP-Module 2]	MOD_Agent_v2.0

1.4. Distributed and Microservices TOE Architectures

For this PP-Configuration, a distributed TOE consists of multiple TOE Components whose instances are separately deployed and collectively provide the TOE security functionality. A TOE Component is a named, logical, separately deployable portion of the TOE that is identified in the ST and mapped to the base cPP, applicable PP-Modules, and relevant SFRs. Each TOE Component and its permitted instance or scaling topology, including any Runtime Replicas, shall be identified in the ST and mapped to the base cPP and, where applicable, to the Server Module, Agent Module, or both according to the role or roles performed by that TOE Component.

The terms Server Application and Agent Application describe TOE Component roles for purposes of this PP-Configuration. They do not imply network client/server directionality, hosting relationship,

privilege model, deployment topology, or that communication is initiated by one role rather than the other. A TOE Component may perform one or both roles if identified in the ST.

For containerized or microservices TOEs, the TOE consists of the application payload components identified in the ST. The container orchestration platform, container runtime, host operating-system kernel, platform-provided operating-system services or userland not delivered as part of a TOE image, service mesh infrastructure, ingress infrastructure, cluster networking, and platform-provided secret or configuration stores are part of the operational environment unless explicitly included in the TOE boundary. Vendor-delivered executable content in a TOE image remains subject to the base cPP's TOE-boundary and composition rules.

The terms *TOE Component*, *TOE Component Instance*, *Runtime Replica*, *Communication Relationship Type*, *Protected Channel Instance*, *Running-Payload Activity*, *SFR Implementation Instance*, and *Component Implementation-Equivalence Class* have the meanings defined by the base cPP. The ST may describe an unambiguous topology or scaling pattern instead of enumerating Runtime Replicas, but shall separately identify every application artifact and every difference in security-relevant configuration, role, interface, relationship, or platform service that can affect an SFR or EA.

1.4.1. Minimum Evaluated Configuration and Permitted Scale-Out

The ST shall identify a minimum evaluated configuration: the set of TOE Components and TOE Component Instances evaluated together to demonstrate that the distributed TOE satisfies the claimed requirements. This is the minimum configuration of interest for the evaluation; it need not be the smallest deployment that could theoretically satisfy the PP-Configuration and may include redundancy needed for the claimed deployment.

The ST may separately identify named TOE Components for which additional Runtime Replica instances are permitted after evaluation. For every such component, the ST shall identify the permitted artifact and security-relevant configuration, the constraints on adding the instance, the expected additional communication relationships, and any effect on SFR allocation, interfaces, identities, trust, management, or update behavior. A permitted additional instance shall preserve the claimed SFR behavior of the corresponding TOE Component and shall not corrupt, bypass, or weaken the security functionality of components already in the TOE.

The operational guidance shall describe how to add a permitted Runtime Replica consistently with the ST. If an additional instance introduces an SFR allocation or completed operation not present in the minimum evaluated configuration, or a difference in security-relevant configuration, interface, identity, trust, platform dependency, or update behavior, the ST shall identify that difference and the evaluator shall cover the additional case. For an additional connection not present in the minimum evaluated configuration, the evaluator shall confirm that it has the security characteristics claimed for that relationship.

Table 3. Containerized TOE/OE Boundary Classification

Item	Default classification	Required treatment
Vendor application payload	TOE	Identify the TOE Component, artifact, role, SFR allocation, configuration, interfaces, and permitted TOE Component Instances, including any Runtime Replicas.
Vendor-provided sidecar, init container, operator, or helper that implements or supports a claimed SFR	TOE content; a separate TOE Component only when it meets the TOE Component definition	Include it in the TOE boundary. Map it into the associated TOE Component, or identify it as a separate TOE Component when it is a named, logical, separately deployable TOE portion. Map its SFR behavior, interfaces, update unit, and evidence in either case.
Libraries, userland utilities, and other executable content delivered with a TOE payload that implement or support a claimed SFR	Part of the associated TOE Component unless identified as a separate TOE Component	Cover the content under the applicable software-inventory, anti-exploitation, vulnerability, and trusted-update activities. Separate packaging alone does not make a library or utility a TOE Component. Identify it separately when it constitutes a named, logical, separately deployable TOE portion and its artifact, security-relevant configuration, role, claimed SFR behavior, relevant platform interface, or update lifecycle requires separate treatment.
Platform-provided base-image operating-system services or userland that do not implement or support a TOE-claimed SFR	Operational environment or platform dependency unless explicitly included	Identify any service or interface on which the TOE relies and apply the base cPP platform-dependency rules. Do not treat every utility in a general-purpose base image as a separate TOE Component or SFR Implementation Instance.
Vendor-supplied deployment manifests, charts, and configuration templates	TOE delivery artifact, configuration data, or guidance; not an executable TOE Component solely by form	Identify security-relevant values and the TOE Components, interfaces, mounts, environment values, and operational-environment services they configure. Classify executable hooks or controllers separately according to the function they perform.
Container runtime, host kernel, and orchestrator	Operational environment unless explicitly included	Document the required service, version or interface, trust assumption, failure behavior, and secure configuration. Do not credit the service with satisfying a TOE-implemented SFR unless that SFR expressly permits platform-provided functionality.

Item	Default classification	Required treatment
Platform service mesh or ingress	Operational environment unless explicitly included	Assess the TOE's dependency and configuration. External termination does not satisfy a TOE channel SFR unless the SFR expressly permits platform invocation; if the vendor-delivered mesh or ingress component performs the TSF, include and map it as a TOE Component.
Platform secret or configuration store	Operational environment unless explicitly included	Identify the consuming TOE Components, source and attachment mechanism, purpose, permissions, failure behavior, and any TOE persistence, transformation, caching, or re-export of the value.
Registry, repository, or external update service	Operational environment unless explicitly included	Identify it as the delivery or authorized-source dependency. Every TOE image or package, applicable signature or integrity check, installation or replacement path, and required removal behavior remains subject to the applicable trusted-update activities. Document any supported rollback behavior without treating it as an additional requirement unless a completed SFR or EA requires it.

The ST shall provide an SFR allocation rationale that separately identifies the claim condition, component allocation, and any permitted operational-environment contribution for each claimed requirement. Component allocation is expressed as all TOE Components, applicable TOE Components that perform the relevant function, or at least one TOE Component. The ST shall describe all distinct intra-TOE Communication Relationship Types. For an in-scope network-mediated Communication Relationship Type, the ST shall identify the mechanisms used to authorize and protect the communication. Repeated relationships may be represented using an unambiguous topology pattern rather than by enumerating every TOE Component Instance pair. A connection limited to the platform loopback interface is not in scope for FTP_DIT_EXT.1 in this version of the cPP and does not require a trusted-channel claim or a separate FTP_DIT_EXT.1 test. This scope boundary does not assert that local users or processes are trusted. For FPT_IPC_EXT.1, the ST shall identify each non-network local inter-process communication mechanism, such as named pipes, Unix domain sockets, shared memory, platform-brokered IPC, or an equivalent mechanism, and the architectural protection relied upon, such as platform isolation, endpoint access control, or object permissions. Detailed implementation or configuration information is required only where an applicable SFR or Evaluation Activity requires it.

1.4.2. SFR Allocation for Distributed TOEs

For a distributed TOE, the SFRs are satisfied by the TOE as a whole; however, not every SFR is necessarily implemented by every TOE Component. The ST author shall separately identify the claim condition, component allocation, and permitted operational-environment contribution for every claimed SFR element. Every deployed TOE Component Instance shall exhibit the claimed SFR behavior

allocated to its TOE Component in its identified configuration. For an *All Components* or *Applicable Components* allocation, deployment multiplicity does not permit one instance to satisfy an allocated requirement on behalf of another instance.

All Components ("All")

Every TOE Component shall independently satisfy the requirement.

Applicable Components

For a claimed SFR, every TOE Component that performs a security-relevant function governed by an SFR element shall satisfy that allocated element. The ST shall identify the components and elements to which the requirement applies. This category does not exclude other TOE Components from the base cPP; it identifies the TOE Components that have concrete implementation obligations because they perform the relevant function, make the relevant selection, or rely on the relevant platform functionality. It does not require every identified component to satisfy every element of a multi-element SFR.

At Least One Component ("One")

At least one TOE Component shall satisfy the requirement on behalf of the TOE. The ST shall identify the component or components that satisfy the requirement and describe how this satisfies the TOE-level claim.

Feature Dependent (condition of applicability)

Feature Dependent describes why an SFR is included in the ST: the distributed TOE claims the relevant feature or selection. An objective SFR is included when the ST claims that objective. This condition does not itself allocate the requirement or reduce evaluation coverage. Once the SFR is included, the allocation in the applicable table controls which TOE Components have concrete implementation obligations. The ST shall identify the TOE Component or components that perform the relevant function, make the relevant selection, or rely on the relevant platform functionality.

Permitted Operational-Environment Contribution (not an allocation category)

The TOE may rely on the operational environment for a function only where the base cPP or PP-Module expressly permits it. The ST shall identify the environmental service, the TOE Component and SFR element that invoke or rely on it, and the required configuration. This does not remove the TOE Component from the SFR mapping or allow the environmental component to satisfy a TOE-implemented requirement.

The claim condition and component allocation are controlled by the following tables and the SFR's operations and application notes; the ST shall not freely reclassify an SFR. *At Least One Component* and a permitted operational-environment contribution may be used only where the applicable table or requirement expressly permits them. Allocation shall be stated at the SFR-element level when different elements have different responsible TOE Components, update actors, or permitted platform dependencies. Evaluation evidence shall cover every TOE Component, artifact, communication path, or function allocated as responsible. Evidence for one TOE Component may be reused only where the ST demonstrates that the relevant implementation, configuration, and execution context are equivalent.

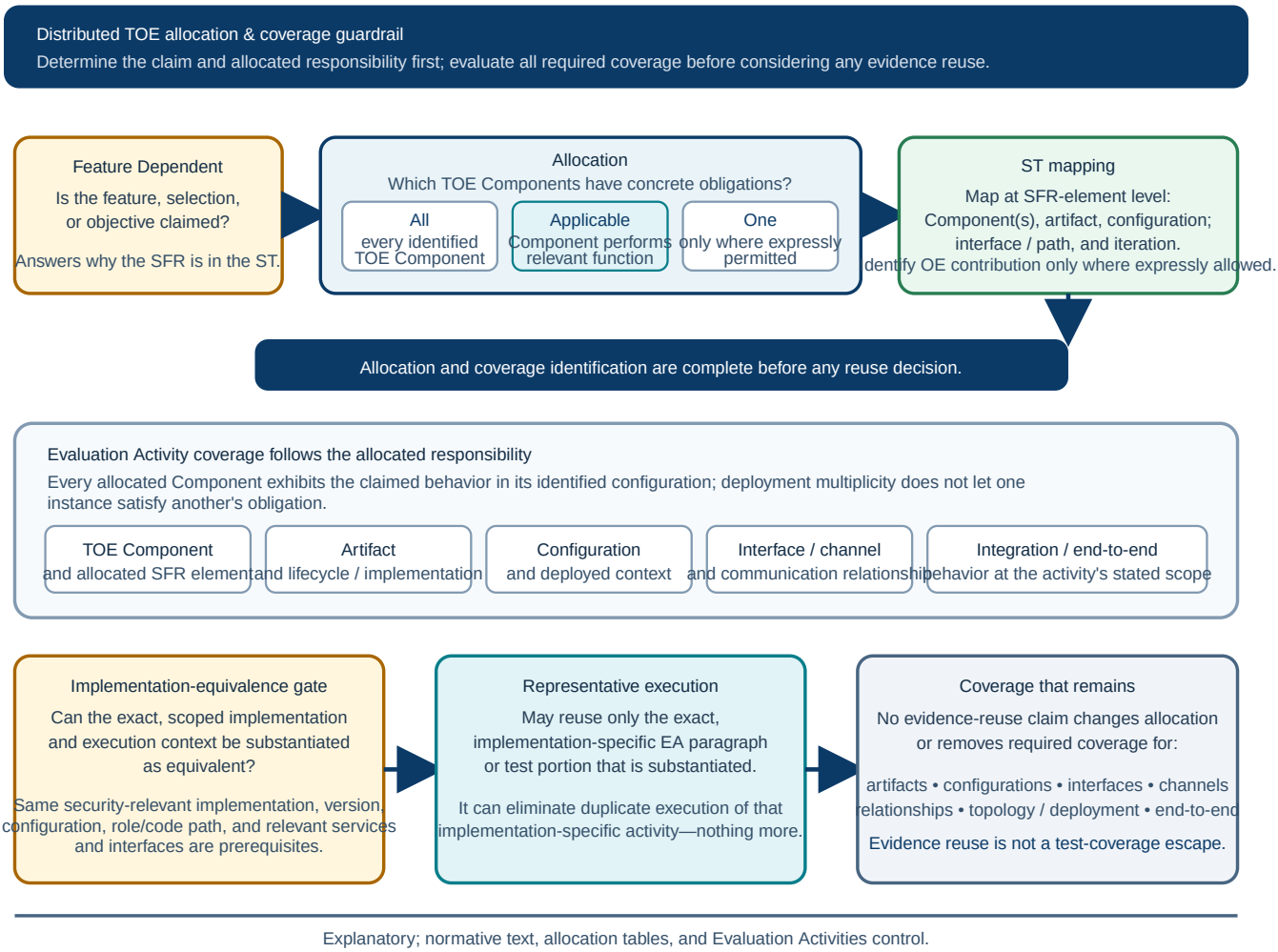

Figure 1. Illustrative claim, allocation, mapping, and retained Evaluation Activity coverage

Rationale: Why Applicable Components is Retained

Feature Dependent and *Applicable Components* answer different questions. *Feature Dependent* says why a requirement is present in the ST: the TOE claims a feature, makes a selection, or claims an objective. *Applicable Components* says which TOE Components must demonstrate the claimed requirement after it is present. Retaining both terms prevents the claim decision from being mistaken for the component-allocation decision.

Applicable Components is not a new SFR type and does not broaden a claimed SFR. It is a component-mapping instruction used to make the standard feature-dependent result explicit: every TOE Component that implements an allocated SFR element is identified and covered. A TOE Component that does not implement that element is not treated as its implementer; conversely, a component that does implement it cannot be omitted merely because another component implements a similar function. The TOE as a whole must still satisfy every claimed SFR, and evidence for one component can be reused only when equivalent security-relevant implementation, configuration, and execution context are demonstrated.

Required ST Allocation Mapping

The ST shall include a mapping for every claimed SFR element or unambiguous set of identically allocated elements. For each mapping entry, the ST shall identify:

- the SFR element and completed iteration, where applicable;
- the claim condition, including a feature or selection, or an objective claim, where applicable;
- the component allocation: *All Components*, *Applicable Components*, or *At Least One Component*;
- the TOE Component or unambiguous set of TOE Components responsible for the element, together with the performed security-relevant function;
- the relevant artifact, security-relevant configuration, interface, communication relationship, or update path where it affects the claim; and
- any permitted operational-environment contribution and the required guidance reference.

The mapping may reference a common TSS description for identically completed SFRs, but it shall remain clear which TOE Components contribute to every claimed SFR and how the TOE as a whole satisfies it. Detailed test evidence may be supplied in the evaluation evidence, but the ST mapping shall be sufficient to determine the components, functions, and relationships that require coverage.

Non-Normative Example: Conditional Update Selection and Component Coverage

Consider a distributed TOE containing a Server Application image and an Agent Application package. The trusted-update selection in FPT_TUD_EXT.2 is feature dependent: if the selection is not claimed, the requirement is not included in the ST. If the selection is claimed, the table's *Applicable Components* allocation applies. The ST identifies every TOE Component that performs update verification, delivery, replacement, or installation integrity functions, together with its artifact and path.

In plain terms, a successful test of the Server image does not prove that the Agent package is protected. If the Server verifies and installs its own image while the Agent verifies and installs its package, each component is applicable only to the update elements it performs. The Server evidence need not prove the Agent's element, and the Agent evidence need not prove the Server's element; together, the evidence must cover all allocated elements. A common registry, build pipeline, base image, or package format does not by itself make the two paths equivalent. This example does not alter the requirement or its Evaluation Activities.

The following table defines the expected allocation for the base cPP SFRs in a distributed TOE.

Table 4. Base cPP SFR Allocation for Distributed TOEs

SFR	Allocation	Distributed TOE guidance
FCS_CKM.1/AK	Applicable Components	Applies to each TOE Component that invokes or implements asymmetric key generation.
FCS_CKM.1/SK	Applicable Components	Applies to each TOE Component that generates symmetric keys.

SFR	Allocation	Distributed TOE guidance
FCS_CKM.2	Applicable Components	Applies to each TOE Component that performs key establishment.
FCS_CKM_EXT.1	Applicable Components	Applies to each TOE Component that invokes or implements key generation services.
FCS_COP.1/Hash	Applicable Components	Applies to each TOE Component that performs hashing for a claimed function.
FCS_COP.1/KeyedHash	Applicable Components	Applies to each TOE Component that performs keyed-hash functions for a claimed function.
FCS_COP.1/SigGen	Applicable Components	Applies to each TOE Component that generates digital signatures.
FCS_COP.1/SigVer	Applicable Components	Applies to each TOE Component that verifies digital signatures, including update verification if performed by that component.
FCS_COP.1/XOF	Applicable Components	Applies to each TOE Component that implements SHAKE as a component of LMS or XMSS signature verification.
FCS_COP.1/SKC	Applicable Components	Applies to each TOE Component that performs encryption or decryption.
FCS_HTTPS_EXT.1	Applicable Components	Applies to each TOE Component that implements HTTPS as a client, server, or server with mutual authentication.
FCS_HTTPS_EXT.2	Applicable Components	Applies to each TOE Component that implements HTTPS with peer certificate authentication behavior covered by this requirement.
FCS_PBKDF_EXT.1	Applicable Components	Applies to each TOE Component that performs password conditioning.
FCS_RBG.1	Applicable Components	Applies to each TOE Component that implements RBG functionality.
FCS_RBG.2	Applicable Components	Applies to each TOE Component that implements RBG functionality using external seeding.
FCS_RBG.3	Applicable Components	Applies to each TOE Component that implements RBG functionality using a single internal noise source.
FCS_RBG.4	Applicable Components	Applies to each TOE Component that implements RBG functionality using multiple internal noise sources.

SFR	Allocation	Distributed TOE guidance
FCS_RBG.5	Applicable Components	Applies to each TOE Component that implements RBG functionality using combined noise sources.
FCS_RBG_EXT.1	Applicable Components	Applies to each TOE Component that invokes platform-provided RBG services, implements RBG functionality, or claims no RBG functionality.
FCS_SNI_EXT.1	Applicable Components	Applies to each TOE Component that creates or uses salts, nonces, or initialization vectors for claimed cryptographic functions.
FCS_STO_EXT.1	Applicable Components	Applies to each TOE Component that persistently stores credentials.
FDP_DAR_EXT.1	Applicable Components	Applies to each TOE Component that stores sensitive application data at rest.
FDP_DEC_EXT.1	All Components	Each TOE Component shall identify and restrict its access to platform resources and sensitive information repositories as required by the base cPP.
FDP_NET_EXT.1	All Components	Each TOE Component shall identify and restrict its inbound and outbound network communications at the actual payload-side TOE interface. When an operational-environment ingress, proxy, or service mesh exposes or terminates a public interface, the ST shall identify both sides of the dependency. The public-facing interface is tested as TOE behavior only when the terminating component is included in the TOE boundary.
FMT_CFG_EXT.1	All Components	Each TOE Component shall satisfy secure-by-default and file-permission requirements for its installed binaries, data, and default credentials.
FMT_MEC_EXT.1	Applicable Components	Applies to each TOE Component that stores or manages configuration options.
FMT_SMF.1	Applicable Components	Applies to each TOE Component that provides security management functions. At least one component shall be identified if management functions are claimed for the TOE.
FPR_ANO_EXT.1	Applicable Components	Applies to each TOE Component that transmits personally identifiable information.

SFR	Allocation	Distributed TOE guidance
FPT_AEX_EXT.1	All Components	Each TOE Component shall satisfy the anti-exploitation requirements applicable to its platform and implementation type.
FPT_API_EXT.1	All Components	Each TOE Component shall use only documented and supported platform APIs.
FPT_API_EXT.2	Applicable Components	Applies to each TOE Component that parses IANA MIME media types covered by the objective requirement.
FPT_IPC_EXT.1 (Objective)	Applicable Components	When the objective is claimed, applies to each TOE Component that uses non-network local inter-process communication with another TOE Component Instance. The ST maps each mechanism, the participating TOE Components, and the architectural protection relied upon.
FPT_FLS.1	Applicable Components	Applies to each TOE Component that must preserve a secure state for the selected failure conditions.
FPT_IDV_EXT.1	All Components	When the objective is claimed, every separately versioned TOE Component shall be represented in the TOE version information. A common version record may be used only when it unambiguously identifies every mapped component and its applicable version.
FPT_LIB_EXT.1	All Components	Each TOE Component shall identify its third-party libraries.
FPT_TST.1	Applicable Components	Applies to each TOE Component that performs TSF self-tests or integrity verification covered by the requirement.
FPT_TUD_EXT.1	Applicable Components	The TOE shall provide trusted update support. The ST shall identify how each separately versioned TOE Component and each distinct container image, package, or update artifact is checked, obtained from an authorized source, delivered, verified, installed or replaced, reported in TOE version information, and removed or rolled back where supported. Running-payload execution does not discharge artifact or lifecycle coverage.

SFR	Allocation	Distributed TOE guidance
FPT_TUD_EXT.2	Applicable Components	Applies to every distinct TOE Component, image, package, update artifact, and delivery or verification path that performs installation or update integrity functions covered by this selection-based requirement. A common registry, build pipeline, base image, or package format alone does not justify representative update-artifact testing.
FTP_DIT_EXT.1	Applicable Components	Applies to each TOE Component that transmits data or sensitive data to another trusted IT product or TOE Component Instance in an in-scope network-mediated relationship, or invokes platform-provided functionality where the SFR permits that protection. A connection limited to the platform loopback interface is not an FTP_DIT_EXT.1 relationship in this version of the cPP and does not require a trusted-channel claim or a separate FTP_DIT_EXT.1 test. This scope boundary does not assert that local users or processes are trusted. If an operational-environment ingress, proxy, or mesh terminates an in-scope connection, the payload-side TOE interface and environmental dependency remain in scope; the public-facing interface is TOE behavior only when the terminating component is in the TOE boundary. Non-network local inter-process communication is addressed by the FPT_IPC_EXT.1 objective.

The following table defines the expected allocation for the Server and Agent PP-Module SFRs in a distributed TOE.

Table 5. Server and Agent Module SFR Allocation for Distributed TOEs

SFR	Allocation	Distributed TOE guidance
FMT_MEC_EXT.1/Server	Applicable Components	Applies to each Server Application TOE Component that stores or manages server configuration data.

SFR	Allocation	Distributed TOE guidance
FMT_SMF.1/Server	Applicable Components	Applies to each Server Application TOE Component that provides management functions. The ST shall identify which TOE Component or Components manage intra-TOE communications, enrollment, or policy.
FPT_AEX_EXT.2/Server	Applicable Components	Each Server Application TOE Component shall be compatible with security features provided by its platform vendor.
FCO_CPC_EXT.1/Agent	Applicable Components	Applies to each Agent Application TOE Component whose instances are represented by endpoint identities in communication pairings that are enabled, disabled, registered, authorized, or otherwise controlled under FCO_CPC_EXT.1/Agent.

If an operational environment component, such as a container orchestration platform, container runtime, service mesh infrastructure, ingress infrastructure, cluster networking, platform-provided secret or configuration store, managed runtime, or application framework, is relied upon to support a claimed SFR, the ST shall identify the dependency. The evaluator assesses the TOE's use of the dependency and the required configuration guidance, but the environmental component is not included in the TOE boundary unless explicitly claimed. The ST shall provide an operational-environment dependency matrix that identifies the service and relevant version or API, affected TOE Components and SFR elements, trust assumption, failure behavior, credentials and configuration, and corresponding guidance requirement. An environmental component may satisfy an SFR only where that SFR expressly permits platform-provided functionality.

For distributed TOEs that rely on managed runtimes or application frameworks, the ST shall identify which TOE Components use each runtime or framework and whether the runtime or framework is included in the TOE, bundled with the TOE Component, or supplied by the operational environment. The SFR allocation rationale shall identify any component-specific runtime or framework dependencies that affect security-relevant behavior such as authentication, authorization, session management, object binding, serialization or deserialization, management endpoint exposure, cryptographic provider selection, trust store handling, configuration loading, native interface exposure, or code loading.

If a TOE container receives credentials, keys, tokens, certificates, or other secrets from an operational environment mechanism, such as a platform secret store, mounted secret volume, injected environment variable, or external secrets provider, the ST shall identify the mechanism, the TOE Components that consume the secrets, the purpose of each secret, and whether the TOE persists, transforms, caches, or re-exports the secret. If the TOE persists or manages the secret after receipt, the applicable base cPP requirements, including FCS_STO_EXT.1, FDP_DAR_EXT.1, FMT_MEC_EXT.1, and related cryptographic requirements, apply to the TOE Component performing that function.

1.4.3. Linux-Container Running-Payload Activities

For a TOE application payload executing in a Linux container, an EA whose subject is behavior of the running payload shall use the Linux-specific method defined by the base cPP and Supporting Document. This does not satisfy or make inapplicable an activity whose subject is the image or update artifact, installation or update workflow, runtime or orchestrator configuration or dependency, mounted or injected data at its source, network policy, or an externally exposed interface or protected channel. Each such activity shall be performed at the artifact, platform, lifecycle, configuration, interface, or channel layer stated by the activity.

1.4.4. Distributed Trusted-Update Coverage

The ST shall provide a compact trusted-update matrix for every separately versioned TOE Component and every distinct update unit or bundle. The matrix shall identify:

- the TOE Component, artifact or image identity, and version identifier or digest;
- the update unit or bundle and any compatibility relationship to other component versions;
- the component or operational-environment service that checks for updates and the authorized source;
- the repository, registry, or other delivery mechanism;
- the component that verifies the signature or integrity protection and the point at which verification occurs;
- the installer, replacement, activation, and failure behavior; and
- required removal behavior and any supported rollback, partial or rolling update, and mixed-version behavior, including an explicit statement when one of those modes is unsupported.

A central TOE service may perform update discovery for the applicable updateable TOE Components when it is mapped as the responsible component, but every changed TOE artifact shall remain covered by an authenticated or integrity-protected chain from the authorized source through verification and installation or replacement. A running-payload version query or observation of executable-change behavior may use the base cPP Linux-container method; authorized-source, signature or integrity, delivery, installation or replacement, and removal activities remain applicable at their stated lifecycle and artifact scope. Rollback, partial or rolling update, and mixed-version behavior are documentation items in this matrix; they are evaluated as functional requirements only when a completed SFR or applicable EA requires them.

1.4.5. Component Implementation-Equivalence Classes and Reuse of Evaluation Evidence

SFR allocation determines which TOE Components must satisfy a requirement. Every deployed TOE Component Instance shall exhibit the claimed behavior of its TOE Component in its identified configuration. Interfaces, communication relationships, artifacts, configurations, and deployment contexts are separate Evaluation Activity coverage targets. Allocation and coverage identification shall

be completed before any reuse of evaluation evidence is considered. Assignment to a class does not change the allocation category or relieve any TOE Component, identified configuration, or deployed instance from satisfying an applicable requirement.

The ST need not iterate an SFR solely because it applies to multiple TOE Components. A single completed SFR statement and common TSS description may apply to multiple TOE Components when the completed operations and claimed security-relevant behavior are the same. Different selections, assignments, or other completed operations require separate SFR iterations. Multiple deployed instances of the same TOE Component are Runtime Replicas when they meet the base cPP definition and may be represented by that TOE Component together with its permitted instance or scaling topology.

The ST shall provide concise traceability that maps every applicable TOE Component and identified security-relevant configuration, where applicable, to the claimed SFR iteration and, where evidence reuse is claimed, to a class identifier and common TSS description. The ST may identify unambiguous sets of TOE Components and configurations rather than repeat one row for every member. A compact table with columns for TOE Component and configuration or set, SFR iteration or set, class identifier, and common TSS reference is sufficient. A single class definition may list multiple SFR iterations when the member SFR Implementation Instances and equivalence basis are identical for all listed SFRs. The detailed implementation-equivalence rationale and the mapping to exact Evaluation Activity portions may be provided as separate evaluation evidence rather than repeated in the ST.

The implementation-equivalence rationale shall identify:

- the class identifier;
- the member SFR Implementation Instances and the TOE Components and identified security-relevant configurations, where applicable, that use them;
- the SFR iterations and exact implementation-specific Evaluation Activity paragraphs or test identifiers for which reuse is proposed;
- the common security-relevant implementation and version, including relevant wrapper or integration behavior, build options, implementation configuration and values relevant to the reused activities, role and exercised code path, cryptographic provider, and immediate platform services and interfaces relevant to the reused activities;
- peripheral differences relevant to, or reasonably capable of affecting, the scoped results and why those differences cannot affect the results; and
- the objective evidence used to verify the membership of every class member.

A shared library name, base image, runtime, source repository, or build pipeline alone is not sufficient when component-specific code or configuration can alter the claimed behavior. The same security-relevant implementation and version, wrapper or integration behavior, build options, implementation configuration and values that affect the reused activities, role and exercised code path, cryptographic provider, and immediate platform services and interfaces relevant to the reused activities are prerequisites for class membership. Channel-specific certificates, keys, trust-anchor values, endpoint identities, addresses, and similar deployed values may differ when they do not alter the reused

implementation behavior and are covered by retained interface- or channel-specific activities. When these prerequisites or the effect of a relevant peripheral difference cannot be substantiated, the affected SFR Implementation Instances may not share representative execution for that scope. Image, runtime, and orchestrator differences remain subject to their own activities; when such a difference can affect a scoped running-payload result, it is also a class-relevant difference.

Class membership shall be re-established when a member's security-relevant implementation version, provider, wrapper or integration behavior, relevant build options, relevant configuration, relevant platform service or interface, or exercised code path changes. When reuse is claimed for an EA incorporated from a Functional Package, the rationale shall identify the package and Supporting Document version and a stable EA paragraph or test identifier.

Where permitted by the applicable Supporting Document, the evaluator may perform the explicitly identified implementation-specific portions of Evaluation Activities using one or more representative class-member SFR Implementation Instances exercised through exact deployed TOE Component Instances. Representative execution does not reduce any internal test repetition or any component-, artifact-, interface-, channel-, configuration-, topology-, deployment-, or end-to-end coverage required by the originating activity. The evaluator shall preserve a coverage mapping and separate verdict for every applicable requirement and activity.

For example, a substantiated TLS implementation used by multiple TOE Components or identified security-relevant configurations may share the complete protocol-implementation test suite only when its SFR Implementation Instances meet the base cPP Component Implementation-Equivalence Class criteria. Each distinct interface and channel remains subject to its required connection, credential, trust, identity, traffic-protection, interruption, recovery, and integration checks. TLS termination performed by an operational-environment service mesh or ingress component is an environmental dependency, not a shared TOE implementation, unless that component is explicitly included in the TOE boundary. An operational-environment termination point cannot be credited as satisfying FTP_DIT_EXT.1 or another TOE-implemented channel requirement unless the applicable SFR expressly permits platform-provided functionality.

1.4.6. Non-Normative Worked Example: Twelve Container Payloads

This example illustrates the documentation and test-compression model. It does not add or change an SFR allocation.

An illustrative TOE has twelve simultaneously deployed TOE Component Instances representing five TOE Components. Each instance is an application payload container. The orchestrator, runtime, host kernel, platform ingress, platform secret store, and registry are in the operational environment. The ST identifies their dependencies using the operational-environment matrix; it does not count them as TOE Components or credit them with TOE-implemented SFRs.

Table 6. Illustrative Component Inventory

TOE Component	Deployed instances	Module role	Principal function
Management API	2	Server + Agent	Administrative API, component registration, and communication policy
Policy Service	2	Server + Agent	Policy storage and decision support
Worker	4	Agent	Processes application work under Management API control
Telemetry Agent	3	Agent	Collects and transmits TOE security telemetry
Update Coordinator	1	Server + Agent	Checks for authorized updates and coordinates the TOE update workflow

The Management API, Policy Service, and Update Coordinator are assigned both Server Application and Agent Application roles because the example applies the Agent Module control model to their communication pairings. These role assignments do not imply network directionality or hosting relationships.

The ST describes five TOE Components and a permitted instance topology rather than repeating the same SFR text twelve times. Multiple instances of a TOE Component are Runtime Replicas when they meet the base cPP definition. An instance with a different image is not a Runtime Replica and shall be separately identified as a TOE Component or explicitly identified configuration, as appropriate. An instance with another difference in security-relevant values, role, interface, identity model, or relevant platform service that can affect an SFR or EA is also not a Runtime Replica and shall be represented separately in the same manner.

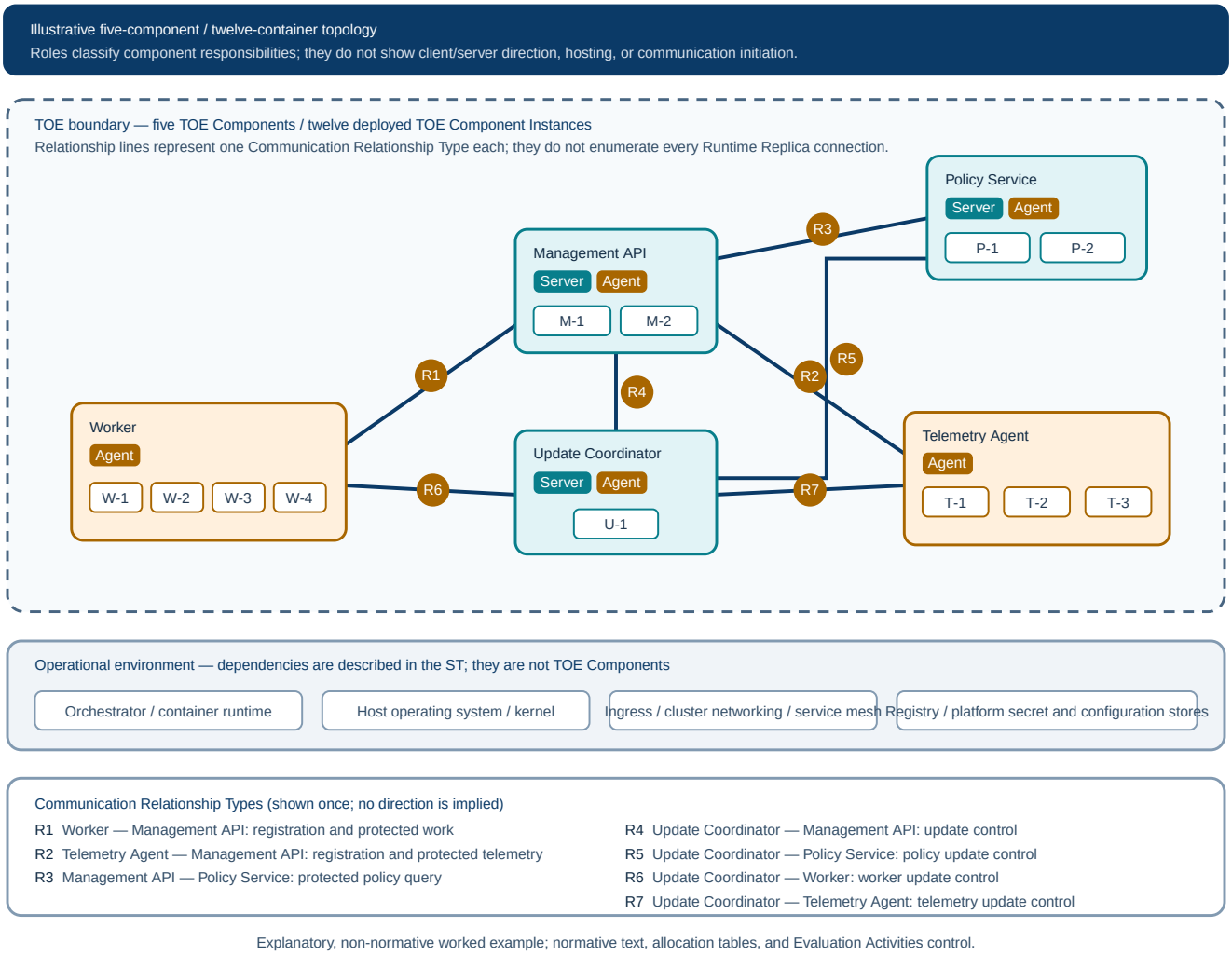

Figure 2. Illustrative twelve-container TOE topology and Communication Relationship Types

Table 7. Illustrative Allocation Extract

SFR or group	Allocation	Illustrative treatment
FPT_AEX_EXT.1, FPT_API_EXT.1, FPT_LIB_EXT.1	All five TOE Components	Each distinct payload artifact satisfies the requirement. A Linux Running-Payload Activity is executed using at least one exact TOE Component Instance representing each TOE Component and each of its identified security-relevant configurations, where applicable, unless a narrower implementation-specific portion is covered by a substantiated class; artifact-specific binary and library evidence remains separate.
FDP_NET_EXT.1	All five TOE Components	The ST identifies each payload-side inbound and outbound interface. The operational-environment ingress dependency is described separately.
FCO_CPC_EXT.1 and FTP_DIT_EXT.1	FCO_CPC_EXT.1: TOE Components assigned an Agent Application role; FTP_DIT_EXT.1: transmitting TOE Components	The ST maps each Communication Relationship Type below to the applicable iteration and retains relationship-, identity-, and channel-specific testing.
FPT_IDV_EXT.1	All five TOE Components	When claimed, the completed identification mechanism records the version identifier or digest of all five separately versioned TOE Components. Querying or reporting those values as part of update behavior is addressed by FPT_TUD_EXT.1.2.
FPT_TUD_EXT.1	Applicable Components	The Update Coordinator may perform discovery for the updateable TOE Components when mapped as the responsible component. The ST allocates element-level responsibilities, including how FPT_TUD_EXT.1.2 reports the version of each separately versioned TOE Component, without changing the controlling Applicable Components category.
FPT_TUD_EXT.2	Applicable Components	Each distinct image and every applicable verification, delivery, replacement, failure, and required removal path remain in the trusted-update matrix. Any supported rollback behavior is documented but is not an additional requirement unless a completed SFR or EA requires it.

Table 8. Illustrative Communication-Relationship Coverage

Initiating TOE Component	Receiving TOE Component	Communication Relationship Type	Retained coverage
Worker	Management API	Worker registration and protected work channel	Registration, administrator enablement, and administrator disablement; deployed identity and trust; connection; plaintext observation; interruption and recovery; and end-to-end work flow
Telemetry Agent	Management API	Telemetry registration and protected telemetry channel	Registration, administrator enablement, and administrator disablement; telemetry-specific authorization; deployed identity and trust; connection; plaintext observation; and interruption and recovery
Management API	Policy Service	Protected policy-query channel	Registration, administrator enablement, and administrator disablement; server-to-server identities; authorization; connection; trust; interruption and recovery; and policy-query integration
Update Coordinator	Management API	Management update-control channel	Registration, administrator enablement, and administrator disablement; authorized component identity; update-control messages; failure behavior; and integration with the Management API artifact lifecycle
Update Coordinator	Policy Service	Policy update-control channel	Registration, administrator enablement, and administrator disablement; authorized component identity; update-control messages; failure behavior; and integration with the Policy Service artifact lifecycle
Update Coordinator	Worker	Worker update-control channel	Registration, administrator enablement, and administrator disablement; authorized component identity; update-control messages; failure behavior; and integration with the Worker artifact lifecycle

Initiating TOE Component	Receiving TOE Component	Communication Relationship Type	Retained coverage
Update Coordinator	Telemetry Agent	Telemetry update-control channel	Registration, administrator enablement, and administrator disablement; authorized component identity; update-control messages; failure behavior; and integration with the Telemetry Agent artifact lifecycle

Runtime Replicas do not create additional Communication Relationship Types solely because there are more connections. However, when a permitted additional instance creates a connection that is not present in the minimum evaluated configuration, the evaluator shall confirm that the connection has the security characteristics claimed for that relationship. Additional SFR or EA coverage is required when an instance has a separately managed identity, trust configuration, interface, placement, failover path, completed SFR operation, platform dependency, or other difference that can affect the result.

The Update Coordinator's own update is shown as a bootstrap or self-update artifact lifecycle in the matrix below. It is not an additional intra-TOE Communication Relationship Type unless another TOE Component Instance performs that update-control communication.

Table 9. Illustrative Trusted-Update Matrix Extract

TOE Component	Update unit and identity	Discovery, source, and verification	Installation, replacement, and documented recovery behavior
Management API	Signed image and digest; component version	Coordinator checks the authorized repository; the identified verifier checks the image signature and digest	Document the actor that replaces both replicas, atomicity or mixed-version policy, failure behavior, and rollback support
Policy Service	Signed image and digest; component version	Same workflow, with artifact-specific positive and negative verification evidence when required by the completed FPT_TUD_EXT.2 selection and EA	Document data compatibility, replacement order, failure behavior, and rollback support
Worker	Signed image and digest; component version	Same workflow, with the four replicas traced to the verified image identity	Document rolling replacement, permitted old/new overlap, failure behavior, and rollback support
Telemetry Agent	Signed image and digest; component version	Same workflow, with the three replicas traced to the verified image identity	Document rolling replacement, telemetry continuity, failure behavior, and rollback support
Update Coordinator	Signed image and digest; component version	Document the bootstrap or self-update authorized source and verifier	Document coordinator replacement, interrupted-update recovery, and rollback support

The matrix documents rollback, rolling replacement, and mixed-version behavior where supported; it does not make those behaviors functional requirements absent a completed SFR or applicable EA. Common verifier implementation testing may use a substantiated Component Implementation-Equivalence Class, but each artifact identity and every distinct delivery and verification path retain the positive, negative, and traceability coverage required by the completed selection and EA.

Suppose the Worker and Telemetry Agent use the same TLS client implementation and version, wrapper, provider, relevant configuration, code path, and platform interface. Their TLS-client SFR Implementation Instances may be placed in one substantiated class and the complete internal protocol vector suite may be executed once on selected exact TOE Component Instances exercising the representative SFR Implementation Instances. The Worker-to-Management and Telemetry-to-Management relationships still receive their separate registration, deployed credential and trust, connection, traffic-protection, interruption, recovery, and integration checks. A Management API TLS

server implementation is not included in the client class merely because it uses the same library.

For Linux Running-Payload Activities, the evaluator may enter or instrument one exact deployed TOE Component Instance representing each applicable TOE Component and each of its identified security-relevant configurations, where applicable, and perform the Linux procedure in its effective execution context. The activity need not be repeated solely for additional Runtime Replicas unless the originating EA expressly requires every deployed instance. If multiple TOE Components or configurations also meet the narrower class criteria for a particular implementation-specific activity, one representative execution may be cited for those mapped SFR Implementation Instances. Image identities and signatures, update paths, runtime and orchestrator controls, mounted or injected sources, and the seven Communication Relationship Types above remain separately covered at their stated layers.

2. Conformance Claims

2.1. CC Statement

To be conformant to this PP-Configuration, an ST must demonstrate Exact Conformance as defined by CC:2022.

2.2. CC Conformance Claims

This PP-Configuration, and its components specified in section 1.3, are conformant to Parts 2 (extended) and 3 (conformant) of Common Criteria CC:2022, Revision 1 [CC].

3. SAR Statement

The set of SARs specified for this PP-Configuration are taken from, and identical to, those specified in the base PP.

3.1. Related Documents

Common Criteria[1]

Table 10. Common Criteria References

[CC1]	Common Criteria for Information Technology Security Evaluation, Part 1: Introduction and General Model, CCMB-2022-11-001, CC:2022, Revision 1, November 2022.
[CC2]	Common Criteria for Information Technology Security Evaluation, Part 2: Security Functional Components, CCMB-2022-11-002, CC:2022, Revision 1, November 2022.

[CC3]	Common Criteria for Information Technology Security Evaluation, Part 3: Security Assurance Components, CCMB-2022-11-003, CC:2022, Revision 1, November 2022.
[CEM]	Common Methodology for Information Technology Security Evaluation, Evaluation Methodology, CCMB-2022-11-006, CC:2022, Revision 1, November 2022.
[ERR]	Errata and Interpretation for CC:2022 (Release 1) and CEM:2022 (Release 1), CCMB-2024-02-001, Version 1.0, 1 February 2024.

[1] For details see http://www.commoncriteriaportal.org/